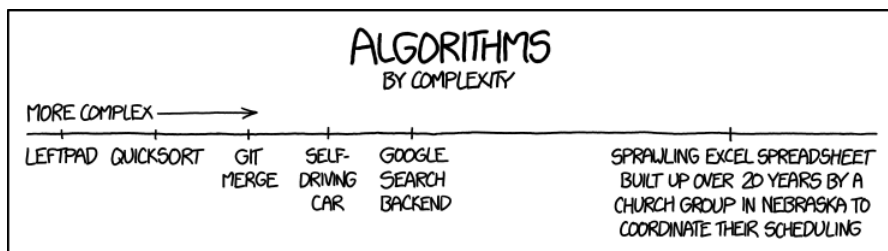


## Chapitre Bonus

# Parlons Complexité

Un extrait de « Algorithmique »



Olivier Caffier



# Table des matières

|          |                                               |          |
|----------|-----------------------------------------------|----------|
| <b>1</b> | <b>Complexité</b>                             | <b>1</b> |
| 1.1      | Complexité temporelle . . . . .               | 1        |
| 1.1.1    | Formalisme . . . . .                          | 1        |
| 1.1.2    | Complexité des boucles . . . . .              | 5        |
| 1.1.3    | Complexité des fonctions récursives . . . . . | 9        |
| 1.1.4    | Théorème maître . . . . .                     | 10       |
| 1.2      | Complexité amortie . . . . .                  | 11       |
| 1.2.1    | Formalisme . . . . .                          | 11       |
| 1.2.2    | Méthode du potentiel . . . . .                | 11       |

# 1 Complexité

On a vu, notamment avec l'exemple de l'exponentiation, que différents algorithmes pouvaient avoir des performances très différentes, tout en résolvant le même problème! La *complexité* est donc l'étude des performances des algorithmes, on l'aborde selon deux critères principaux :

- La *complexité temporelle* décrit le temps de calcul nécessaire à l'exécution de l'algorithme.
- La *complexité spatiale* mesure l'espace mémoire utilisé pour les données de travail de l'algorithme.

Ces notions dépendent bien souvent des entrées.

## 1.1 Complexité temporelle

### 1.1.1 Formalisme

#### Définition - Complexité temporelle

La *complexité temporelle* d'un algorithme est le nombre d'opérations élémentaires qu'il effectue en fonction de la taille de l'entrée.

Une opération élémentaire peut être :

- une opération arithmétique;
- une affectation;
- une condition (i.e une opération booléenne);
- un accès à un tableau;
- un accès à la tête d'une liste;

EXEMPLE 1.1 Prenons l'exemple de la recherche d'un élément dans un tableau.

---

#### Algo - Recherche d'un élément dans un tableau

---

**Require:** *tab* de taille *n*, *elt*

```
for i ← 0 to n do
  if tab[i] = elt then
    return i
return -1
```

---

Ici, on peut déjà encadrer le nombre de comparaisons effectuées, en fonction de la taille du tableau pris en entrée :

- Dans “le meilleur des cas” : l’élément se trouve au début du tableau, on ne fait que une seule comparaison.
- Dans “le pire des cas” : l’élément n’est pas dans le tableau, on le parcourt alors en entier pour s’en rendre compte, on a donc réalisé  $n$  comparaisons.

Ainsi, en notant  $C$  le nombre de comparaisons, on a l’encadrement suivant :

$$1 \leq C \leq n$$

On en déduit de par un encadrement, cette définition :

#### Définition - pire cas / meilleur cas / cas moyen

On distingue alors trois notions pour la complexité d’un algorithme, en fonction d’une taille donnée de l’entrée :

- *Pire cas* : la complexité maximale pouvant être atteinte avec une entrée de la taille donnée. Il s’agit du cas le plus défavorable, sa valeur ne sera à coup sûr jamais dépassée.
- *Meilleur cas* : la complexité minimale pouvant être atteinte avec une entrée de la taille donnée. Cette valeur nous renseigne sur le mieux que puisse faire l’algorithme, sur les entrées les plus favorables.
- *Cas moyen* : la moyenne des complexités pour toutes les entrées possibles d’une taille donnée. Cette valeur donne une indication de ce qu’on peut espérer pour des entrées aléatoires.

#### Définition - Ordre de grandeur et complexité asymptotique

Dans l’étude de la complexité d’un algorithme, on s’intéresse principalement à la *complexité asymptotique* : i.e à la manière dont la complexité évolue pour de grandes valeurs de la taille de l’entrée.

Le plus souvent, on ne cherche pas non plus à avoir une expression exacte de la complexité, mais une simple indication de *l’ordre de grandeur*.

Les *notations de Landau* donnent différents manières de décrire un ordre de grandeur :

**Définition - Notations de Landau**

Soient une fonction  $f, g : \mathbb{N} \rightarrow \mathbb{N}$ , on a alors les notations suivantes :

➤  $g(n) = \mathcal{O}(f(n))$  si :

$$\exists K \in \mathbb{R}_+, \exists n_0 \in \mathbb{N} \text{ tq. } \forall n \geq n_0, g(n) \leq K f(n)$$

⇒ Pire des cas.

➤  $g(n) = \Omega(f(n))$  si :

$$\exists k \in \mathbb{R}_+, \exists n_0 \in \mathbb{N} \text{ tq. } \forall n \geq n_0, k f(n) \leq g(n)$$

⇒ Meilleur des cas.

➤  $g(n) = \Theta(f(n))$  si :

$$\exists k_1, k_2 \in \mathbb{R}_+, \exists n_0 \in \mathbb{N} \text{ tq. } \forall n \geq n_0, k_1 f(n) \leq g(n) \leq k_2 f(n)$$

⇒ Cas moyen.

On dénote alors quelques fonctions de complexité que l'on croise fréquemment (en notant  $n$  la taille de l'entrée) :

**Profils de complexité**

| Complexité  | Appellation    |
|-------------|----------------|
| 1           | constante      |
| $\log(n)$   | logarithmique  |
| $n$         | linéaire       |
| $n \log(n)$ | linéarithmique |
| $n^2$       | quadratique    |
| $n^3$       | cubique        |
| $2^n$       | exponentielle  |

EXEMPLE 1.2 En reprenant notre exemple, on a alors que

$$C(n) = \mathcal{O}(n) \text{ et } C(n) = \Omega(1)$$

⇒ La complexité de cet algorithme est donc linéaire dans le pire des cas, constante dans le meilleur.

### Proposition - Règles de calcul sur les $\mathcal{O}$

Soient  $f, g, h_1, h_2 : \mathbb{N} \rightarrow \mathbb{N}$ .

➤ Si  $f(n) = \mathcal{O}(1)$  et  $g(n) = \mathcal{O}(h_2(n))$ , alors

$$f(n) + g(n) = \mathcal{O}(h_2(n)) \text{ et } f(n) \times g(n) = \mathcal{O}(h_2(n))$$

➤ Si  $f(n) = \mathcal{O}(h_1(n))$  et  $g(n) = \mathcal{O}(h_1(n))$ , alors

$$f(n) + g(n) = \mathcal{O}(h_1(n))$$

➤ Si  $f(n) = \mathcal{O}(h_1(n))$  et  $g(n) = \mathcal{O}(h_2(n))$ , alors

$$f(n) + g(n) = \mathcal{O}(h_1(n) + h_2(n))$$

➤ Si  $f(n) = \mathcal{O}(h_1(n))$  et  $g(n) = \mathcal{O}(h_2(n))$ , alors

$$f(n) \times g(n) = \mathcal{O}(h_1(n) \times h_2(n))$$

### 1.1.2 Complexité des boucles

#### ➤ Pour les boucles simples

Généralement, l'entête de la boucle nous donne directement le nombre de tours, il nous suffit juste de voir cela comme une somme!

EXEMPLE 2. Considérons une boucle assez quelconque écrite en langage C :

```
1 for (int i = 0 ; i < n ; i++){  
2     // Op. quelconques...  
3 }
```

Ainsi, en notant  $C_i$  le nombre d'opérations élémentaires faites au tour de boucle  $i$ , on a que le nombre total d'opérations est :

$$C(n) = \sum_{i=0}^{n-1} C_i$$

Et donc, si  $\forall i \in \llbracket 0; n-1 \rrbracket$ ,  $C_i = \mathcal{O}(1)$ , on a :

$$\begin{aligned} C(n) &= \sum_{i=0}^{n-1} C_i \\ &= \sum_{i=0}^{n-1} \mathcal{O}(1) \\ &= \mathcal{O}(1) \times \sum_{i=0}^{n-1} 1 \\ &= \mathcal{O}(1) \times n \end{aligned}$$

et  $n$  est tout sauf une constante, considérons alors cela comme un produit entre deux fonctions.

Finalement, selon nos règles de calculs, on a :

$$C(n) = \mathcal{O}(n)$$

 **Méfiez-vous!** Des fois, les  $C_i$  ne seront pas en  $\mathcal{O}(1)$  et disposeront même de “coûts cachés” en faisant appel à des fonctions auxiliaires par exemple.

#### ➤ Pour les boucles conditionnelles

Le nombre de tours d'une boucle **while** est déterminé en comparant la condition d'arrêt et l'évolution des différents éléments intervenant dans cette condition d'arrêt :

```

1  int i = n;
2  while (i > 0){
3      // Op. quelconques ...
4      i --;
5  }

```

On peut alors, dans certains cas, retrouver les mêmes schémas que pour les boucles simples. Néanmoins, ce ne sera pas toujours le cas :

EXEMPLE 3. Considérons le programme suivant :

```

1  int a = 0;
2  int b = 1;
3  while (b < n){
4      b = a + b ;
5      a = b - a ;
6  }

```

Ici, le nombre de tours de boucle est lié au nombre de nombres de Fibonacci dans l'intervalle  $[0; n[$ . Ce nombre peut être calculé, mais cela repose sur une analyse mathématique non triviale (exo).

➤ **Pour les boucles consécutives** on additionnera juste les complexités des deux boucles (cf. règles de calculs).

### ➤ Pour les boucles emboîtées

On calcule d'abord la complexité de l'intérieur de la boucle, puis on se ramène à une somme classique comme pour une boucle simple.

EXEMPLE 4. Considérons la double boucle suivante :

```
1 for (int i = 0; i < n ; i++ ){  
2   for (int j = 0; j < n ; j++ ){  
3     // Op. quelconques  
4   }  
5 }
```

Ainsi, pour  $i, j \in \llbracket 0; n-1 \rrbracket$ , on note  $C_{i,j}$  le nombre d'opérations élémentaires réalisées dans la double boucle au tour  $(i, j)$ .

Alors, on a

$$C(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} C_{i,j}$$

Et donc, si  $\forall i, j \in \llbracket 0; n-1 \rrbracket, C_{i,j} = \mathcal{O}(1)$  :

$$\begin{aligned} C(n) &= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} C_{i,j} \\ &= \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \mathcal{O}(1) \\ &= \mathcal{O}(1) \times \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} 1 \\ &= \mathcal{O}(1) \times n^2 \\ &= \mathcal{O}(n^2) \end{aligned}$$

EXEMPLE 5. On rappelle l'algorithme de tri par sélection :

Ainsi, à l'intérieur de la boucle interne, on a que des opérations en  $\mathcal{O}(1)$ . La boucle externe comprend la boucle interne et un appel à la fonction **swap**, supposé en  $\mathcal{O}(1)$  également.

---

**Algo** - Tri par sélection

---

**Require:** `tab` de taille  $n \geq 2$

```
for i ← 0 to n-2 do
  j_min ← i
  for j ← i+1 to n-1 do
    if tab[j] < tab[j_min] then
      j_min ← j
  swap(tab, i, j_min)
```

---

On a donc :

$$\begin{aligned} C(n) &= \sum_{i=0}^{n-2} \left( \mathcal{O}(1) + \sum_{j=i+1}^{n-1} \mathcal{O}(1) \right) \\ &= \sum_{i=0}^{n-2} \left( \mathcal{O}(1) + \mathcal{O}(n-i-1) \right) \\ &= \mathcal{O}(n^2) \end{aligned}$$

Néanmoins, si l'on veut compter exactement le nombre de comparaisons effectuées, on a :

$$\begin{aligned} C(n) &= \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 \\ &= \sum_{i=0}^{n-2} n-i-1 \\ &= \frac{n(n-1)}{2} \end{aligned}$$

### Quelques sommes fréquentes

$$\sum_{k=0}^n k = \frac{n(n+1)}{2}$$

$$\sum_{k=0}^n 2^k = 2^{n+1} - 1$$

$$\sum_{k=0}^n k^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{k=1}^n \frac{1}{k} \sim \ln(n)$$

$$\sum_{k=0}^n k^3 = \frac{n^2(n+1)^2}{4}$$

$$\sum_{k=1}^n \log(k) \sim n \log(n)$$

### 1.1.3 Complexité des fonctions récursives

L'astuce systématique est de **se ramener à une formule de récurrence** concernant le nombre d'opérations réalisées pour une certaine taille d'entrée.

EXEMPLE 6. Prenons la version récursive de l'algorithme "factorielle" :

```

1  let rec factorielle n =
2    if n = 0 then 1
3    else n * (factorielle (n-1))

```

Alors, en notant  $C(n)$  le nombre d'opérations élémentaires réalisées par cet algorithme pour l'entier  $n$ , on a :

$$\begin{cases} C(0) = 1 \\ C(n+1) = 1 + C(n) \end{cases}$$

(on oublie pas le coût de la multiplication!)

Ainsi, on retrouve l'expression d'une suite arithmétique, d'où

$$C(n) = n + 1 = \mathcal{O}(n)$$

On retrouve une complexité "pire cas" linéaire.

EXEMPLE 8. Reprenons l'algorithme de la recherche dichotomique dans sa version récursive (exemple 9), on a ainsi (en posant  $n = f - d$ ) :

$$C(n) \leq 1 + C(\lfloor \frac{n}{2} \rfloor)$$

Ici, la technique (ULTRA-CLASSIQUE) est de considérer les entrées de taille de puissance de 2, i.e on prend  $n = 2^k$  avec  $k \in \mathbb{N}$ , alors :

$$C(2^k) \leq 1 + C(2^{k-1})$$

Ainsi, en définissant  $C_k = C(2^k)$ , on a

$$\forall k \in \mathbb{N}, C_k = C(2^k) \leq k$$

Finalement, en passant au log, on a :

$$C(n) \leq \log_2 n$$

$$\text{d'où } C(n) = \mathcal{O}(\log(n))$$

**Rmq** : Ici, la majoration est faite par  $1 + C(\lfloor \frac{n}{2} \rfloor)$  mais parfois la majoration sera moins évidente à trouver, il faudra d'abord trouver les complexités des fonctions auxiliaires (ex : tri fusion)...

EXEMPLE 9. On retrouve la même complexité pour l'exponentiation rapide grâce au même procédé :

$$C(n) = \mathcal{O}(\log n)$$

#### 1.1.4 Théorème maître

##### Théorème (HP) - Théorème maître (avec notation de Landau)

Supposons la relation de récurrence suivante :

$$C(n) = aC(\lfloor \frac{n}{b} \rfloor) + \mathcal{O}(n^\alpha)$$

On a :

- Si  $\alpha < \log_b a$  alors  $C(n) = \mathcal{O}(n^{\log_b a})$
- Si  $\alpha = \log_b a$  alors  $C(n) = \mathcal{O}(n^\alpha \log_b n)$
- Si  $\alpha > \log_b a$  alors  $C(n) = \mathcal{O}(n^\alpha)$

## 1.2 Complexité amortie

### 1.2.1 Formalisme

#### Définition - Complexité amortie

La *complexité amortie* d'un algorithme est une analyse de complexité lissée sur une séquence d'invocations successives de cet algorithme.

EXEMPLE 10. Dans la structure *union-find*, lorsque l'on optimise la fonction *find* en utilisant la compression des chemins : on garde la même complexité pire cas au départ, mais après une série d'appels successifs on peut dire que *son coût a été amorti* (on a raccourci le chemin entre les éléments et leur représentant).

**Rmq** : On utilise notamment la complexité amortie pour des algorithmes regroupant les 3 caractéristiques suivantes :

- avoir une faible complexité sur de nombreuses entrées ;
- être potentiellement très coûteux sur certaines entrées ;
- être tel que, dans toute utilisation répétée de l'algorithme, les entrées coûteuses surviennent suffisamment rarement pour que le coût moyen reste faible ;

### 1.2.2 Méthode du potentiel

Pour conduire une analyse de complexité amortie, la *méthode du potentiel*, aussi appelée *méthode du physicien*, consiste à associer à chaque entrée possible un nombre appelé *potentiel*. Comme en physique, ce « potentiel » dénote quelque chose (ici un coût) qui est latent dans l'entrée, i.e présent mais pas encore réalisé.

#### Définition - Potentiel

Une *fonction de potentiel* est une application  $\Phi$  qui associe à chaque valeur d'entrée possible  $x$  d'un algorithme, une valeur positive ou nulle  $\Phi(x)$ , alors appelée *potentiel*.

➤ Le comportement typique d'un algorithme ayant de bonnes propriétés de complexité amortie alterne entre des opérations de faible coût faisant monter progressivement le potentiel, et des opérations de coût élevé associées à une brusque diminution du potentiel (si la fonction de potentiel nous indique qu'une entrée a potentiellement un faible coût).

Dans l'analyse de complexité amortie, au lieu de ne s'intéresser qu'au coût effectif de chaque opération, on tient compte également de la *variation de potentiel*.

#### Définition - Coût amorti

Considérons une opération  $op$ , dont l'exécution produit une sortie  $x_s$  à partir d'une entrée  $x_e$ .

Le *coût réel* de  $op$ , noté  $C$ , est la complexité temporelle de son exécution.

Son *coût amorti*, noté  $A$ , y ajoute la variation de potentiel de l'entrée à la sortie :

$$A = C + \Phi(x_s) - \Phi(x_e)$$

➤ On peut montrer que, sur une séquence d'opérations enchaînées commençant avec un potentiel nul, le coût réel ne dépasse jamais le coût amorti.

➤ Ce qui caractérise une « séquence d'opérations consécutives » est que chaque nouvelle opération prend comme entrée la sortie de l'opération précédent. Autrement dit, chaque opération repart de l'état, et donc du potentiel, laissé par la précédente.

#### Théorème - Amortissement

Considérons une succession de  $n$  opérations :

$$x_0 \xrightarrow{op_1} x_1 \xrightarrow{op_2} \dots \xrightarrow{op_n} x_n$$

à partir d'une entrée  $x_0$  pour laquelle  $\Phi(x_0) = 0$ .

En notant  $C_i$  le coût réel de  $op_i$  et  $A_i$  son coût amorti, on a alors la relation suivante :

$$\sum_{i=1}^n C_i \leq \sum_{i=1}^n A_i$$

DÉMONSTRATION.

On a, par définition, pour tout  $i \in \llbracket 1; n \rrbracket$  :

$$A_i = C_i + \Phi(x_i) - \Phi(x_{i-1})$$

Donc, en sommant, on obtient une somme télescopique :

$$\begin{aligned} \sum_{i=1}^n A_i &= \sum_{i=1}^n \left( C_i + \Phi(x_i) - \Phi(x_{i-1}) \right) \\ &= \sum_{i=1}^n C_i + \underbrace{\Phi(x_n)}_{\geq 0} - \underbrace{\Phi(x_0)}_{=0} \end{aligned}$$

D'où

$$\sum_{i=1}^n A_i \geq \sum_{i=1}^n C_i$$

EXEMPLE 11. Considérons une pile munie des opérations suivantes :

- $\text{PUSH}(S, x)$  : empile un objet  $x$  sur la pile  $S$ ;
- $\text{POP}(S)$  : dépile le sommet de la pile  $S$  et retourne l'objet dépilé;
- $\text{MULTIPOP}(S, k)$  : dépile au plus  $k$  objets de la pile  $S$ ;

On peut par exemple voir l'opération  $\text{MULTIPOP}$  comme ci-dessous :

---

**Algo** -  $\text{MULTIPOP}(S, k)$

---

**while**  $S \neq \emptyset$  **and**  $k \neq 0$  **do**

$x \leftarrow \text{POP}(S)$

$k \leftarrow k - 1$

---

Calculons la complexité de chacune des 3 opérations et déduisons-en (avec la méthode du potentiel) le coût amorti pour une suite de  $n$  opérations  $\text{PUSH}$ ,  $\text{POP}$  et  $\text{MULTIPOP}$  sur une pile initialement vide.

➤ Les opérations  $\text{PUSH}$  et  $\text{POP}$  se font en  $\mathcal{O}(1)$  et  $\text{MULTIPOP}$  en  $\mathcal{O}(\min\{|S|, k\})$ .

➤ On définit la fonction de potentiel  $\Phi$  comme étant le nombre d'objets présents dans la pile. Ainsi, pour la pile vide (et donc pour  $x_0$  l'entrée initiale), on aura bien  $\Phi(x_0) = 0$ .

D'autre part, comme le nombre d'objets dans la pile est toujours positif, on aura bien  $\Phi(x) \geq 0$  pour tout  $x$  : il s'agit donc bien d'une fonction de potentiel.

Calculons les différences de potentiel en fonction des différentes opérations :

- PUSH : la différence de potentiel après une opération PUSH est  $\Phi(x_{i+1}) - \Phi(x_i) = (|S|_i + 1) - |S|_i = 1$ . On en déduit un coût amorti :

$$\begin{aligned} A_{i+1} &= C_{i+1} + \Phi(x_{i+1}) - \Phi(x_i) \\ &= \underbrace{C_{i+1}}_{=1} + 1 \\ &= 2 \end{aligned}$$

- POP : la différence de potentiel après une opération POP est  $\Phi(x_{i+1}) - \Phi(x_i) = (|S|_i - 1) - |S|_i = -1$ . On en déduit un coût amorti :

$$\begin{aligned} A_{i+1} &= \underbrace{C_{i+1}}_{=1} - 1 \\ &= 0 \end{aligned}$$

- MULTIPOP( $S, k$ ) : la différence de potentiel après une opération MULTIPOP( $S, k$ ) est  $\Phi(x_{i+1}) - \Phi(x_i) = -k'$  avec  $k' = \min\{|S|, k\}$ . On en déduit un coût amorti :

$$\begin{aligned} A_{i+1} &= \underbrace{C_{i+1}}_{=k'} - k' \\ &= 0 \end{aligned}$$

➤ **Le coût amorti de chacune de ces opérations est en  $\mathcal{O}(1)$** , ce qui ne change rien pour les deux premières vis à vis de la complexité pire cas, mais cela change beaucoup pour MULTIPOP!

➤ On a donc un coût total amorti en  $\mathcal{O}(n)$  pour  $n$  opérations consécutives.

EXEMPLE 12. Soient  $n$  opérations *enfiler/défiler* dans un ordre quelconque. Soient  $f_0, \dots, f_n$  les états de la file tq :

- $f_0$  correspond à la file vide
- $\forall i \in \llbracket 1; n \rrbracket, f_i$  correspond à l'état de la file à l'étape  $i$
- En particulier,  $f_n$  est l'état de fin

Pour tout  $i \in \llbracket 0, n-1 \rrbracket$ , on pose  $C_i$  la complexité pour passer de  $f_i$  à  $f_{i+1}$ . On veut estimer la **complexité amortie**, i.e on veut estimer

$$\frac{\sum_{i=0}^{n-1} C_i}{n}$$

Soit  $\Phi : \text{file} \rightarrow \mathbb{N}^*$  qui associe pour toute file  $f_i$  la taille de la pile  $p_{\text{arrière}}$ .

### Une file avec deux piles

On a deux piles : avant et arrière. Le premier élément de la pile avant est le premier élément de la file et le premier élément de la pile arrière est le dernier élément de la file. On décrit les opérations suivantes :

- **Enfiler** : empiler sur la pile arrière
- **Défiler** :
  - si la pile avant est non-vide : on dépile la pile avant
  - sinon : on transpose le contenu de la pile arrière dans la pile avant, puis on dépile la pile avant.
- **Tête** : idem

### Proposition - Complexités de l'implémentation d'une file avec deux piles

- **Enfiler** :  $\mathcal{O}(1)$
- **Défiler** : au mieux  $\mathcal{O}(1)$ , au pire  $\mathcal{O}(n)$
- **Tête** : idem

On va alors calculer  $C_i + \Phi(f_{i+1}) - \Phi(f_i)$  :

- 1<sup>er</sup> cas : on enfiler donc  $C_i = 1, \Phi(f_{i+1}) - \Phi(f_i) = 1$  donc au total :  $C_i + \Phi(f_{i+1}) - \Phi(f_i) = 2$
- 2<sup>ème</sup> cas : on défile sans retourner la pile arrière donc  $C_i = 1, \Phi(f_{i+1}) = \Phi(f_i)$  donc au total  $C_i + \Phi(f_{i+1}) - \Phi(f_i) = 1$
- 3<sup>ème</sup> cas : on défile en retournant  $p_{\text{arrière}}$ , ainsi  $C_i = \Phi(f_i) = \text{longueur de } p_{\text{arrière}}$  et  $\Phi(f_{i+1}) = 0$ . Donc  $C_i + \Phi(f_{i+1}) - \Phi(f_i) = 0$ .

Ainsi, on en déduit que pour tout  $i$  :

$$0 \leq C_i + \Phi(f_{i+1}) - \Phi(f_i) \leq 2$$

Donc

$$0 \leq \sum_{i=0}^{n-1} A_i \leq 2n$$

$$0 \leq \frac{\sum_{i=0}^{n-1} A_i}{n} \leq 2$$

d'où le résultat voulu.